

Payment Exception & Reconciliation Workflow Redesign

1. Business Case

The institution currently reconciles transactions manually across an internal ledger, a payment gateway feed, and an external clearing-house settlement file. This manual process creates two simultaneous, opposing costs: (a) avoidable delay in book closure and operational throughput, since every mismatch — however trivial — requires the same manual investigation, and (b) avoidable audit/compliance risk, since the current process has no standardized, logged rationale for how a given mismatch was resolved. A rules-based matching framework with aging-based escalation resolves both by auto-resolving low-risk, clearly-defined matches instantly, while ensuring genuine exceptions receive escalating scrutiny the longer they remain unresolved — with every resolution path, automated or manual, equally auditable.

2. Current-State Pain Points

- Manual CSV export and Excel-based lookup across three disconnected data sources (assumption: this is the as-is state for this project's fictional institution)
- No standardized matching logic — resolution approach varies by which operator handles a given exception
- No formal audit trail distinguishing an automatically-cleared minor variance from a manually-investigated genuine break
- No aging-based prioritization — a break that is 2 hours old and one that is 80 hours old receive the same handling approach
- Estimated manual effort: ~3.5 hours/day per operator (planning assumption, not a measurement — see Problem Statement; not validated against real data anywhere in this project)
- Estimated unresolved-past-72h rate: ~15% of exceptions (planning assumption, not a measurement — same caveat)

3. Target-State Goals

- Transactions are matched automatically using a transparent, deterministic rule set (exact match, tolerance match, exception)
- Exceptions that do not auto-resolve are bucketed by age (0–24h / 24–72h / >72h) with escalating visibility and required action at each bucket
- Every resolution — automated or manual — is logged with who/what/when/why, satisfying both Finance's need for throughput visibility and Compliance's need for investigative traceability

- The tolerance threshold separating an auto-resolved match from a flagged exception is documented as a deliberate, justified parameter — not an arbitrary number, and not implied to be a regulatory requirement

4. Success Metrics (stated as design targets, not measured outcomes)

Metric	Target (design intent, not measured)	Source
Auto-resolution rate for exact/tolerance matches	Majority of daily transaction volume resolved without manual touch	FRD matching rules — design specification only
Average time-to-resolution by aging bucket	Faster resolution for 0–24h bucket than current uniform manual baseline	FRD aging-bucket logic — design specification only
Audit trail completeness	100% of resolutions (automated or manual) carry a logged reason code	FRD audit-log requirement
Tolerance threshold justification	Documented rationale (FX rounding/spread variance), not claimed as a regulatory or industry-standard figure	FRD — stated design rationale

Note on this table, carried over from this project's integrity rule: unlike Project A, none of the figures above will ever be tested against real data in this project. They describe what the design is *intended* to achieve, not what was observed. No future stage of this project supersedes this table with measured results.

5. Assumptions & Constraints

- This is a portfolio/demonstration project; no real production data, ledger system, or clearing-house feed exists
- The ~3.5 hours/day and ~15% unresolved-past-72h figures are planning assumptions for sizing the business case only, explicitly labeled as such everywhere they appear, and are never validated or revised by real data anywhere in this project
- The matching tolerance baseline (defined in the FRD) is a pragmatic starting parameter intended to account for routine FX rounding and settlement-timing variance — it is not derived from any regulatory standard and must not be cited as one
- No SQL query in this project is actually executed; the FRD specifies query logic in enough detail that an engineering team could build it, but no script is run and no output is generated
- No Power BI dashboard is built for this phase — reporting and visualization are explicitly a Phase 2 deliverable (see Section 6)

- No mock dataset is generated for this project, unlike Project A

6. Out of Scope

- No SQL execution on real or mock data
- No Power BI dashboard or any other reporting/visualization layer — **Phase 2 deliverable, out of current scope**
- No mock dataset construction
- No actual integration with a real ledger, payment gateway, or clearing-house system